



MUST-KNOW ANGULAR CONCEPTS

SWIPE FOR MORE



Introduction



Angular is a **full-featured frontend framework** built by Google for creating large, maintainable, and high-performance web applications. Mastering Angular requires understanding both its **architecture** and **core building blocks**.

1. Angular Architecture Overview

Angular follows a **component-based architecture** and is built around:

- Components
- Modules
- Templates
- Services
- Dependency Injection
- RxJS & Observables



Angular applications are **opinionated**, meaning the framework enforces structure and best practices.

2. Angular Modules (NgModule)

Modules organize an Angular application into cohesive blocks of functionality.

Types of Modules

- Root Module (AppModule)
- Feature Modules
- Shared Modules
- Core Module

See next slide



Example:

```
● ● ●  
@NgModule({  
  declarations: [AppComponent],  
  imports: [BrowserModule],  
  bootstrap: [AppComponent]  
})  
export class AppModule {}
```

Why it matters

- Improves maintainability
- Enables lazy loading
- Encourages separation of concerns

See next slide

3. Components (The Heart of Angular)



Components control a part of the UI.

A Component Has:

- TypeScript class (logic)
- HTML template (view)
- CSS/SCSS (styling)

```
@Component({  
  selector: 'app-header',  
  templateUrl: './header.component.html'  
})  
export class HeaderComponent {}
```

Key ideas

- Reusable UI blocks
- Parent-child communication
- Lifecycle hooks

4. Templates & Data Binding



Angular templates extend HTML with Angular syntax.

Types of Data Binding

Interpolation

```
<h1>{{ title }}</h1>
```

Property Binding

```
<img [src]="imageUrl">
```

Event Binding

```
<button (click)="save()">Save</button>
```



Two-Way Binding



```
<input [(ngModel)]="username">
```

5. Directives

Directives manipulate the DOM.

Types of Directives

Structural Directives

Change the DOM structure:

- *ngIf
- *ngFor
- *ngSwitch



```
<div *ngIf="isLoggedIn">Welcome</div>
```



Attribute Directives

Change appearance or behavior:

- `ngClass`
- `ngStyle`

6. Services & Dependency Injection (DI)

Services handle **business logic**, API calls, and shared data.



```
@Injectable({
  providedIn: 'root'
})
export class UserService {
  getUsers() {}
}
```




Dependency Injection

Angular automatically provides services to components:



```
constructor(private userService: UserService) {}
```

Why DI is powerful

- Loose coupling
- Easier testing
- Centralized logic

7. Routing & Navigation

Angular Router enables **Single Page Application (SPA)** navigation.



Route Configuration

```
const routes: Routes = [  
  { path: 'login', component: LoginComponent }  
];
```

Router Features

- Route parameters
- Guards
- Lazy loading
- Child routes

8. Angular Lifecycle Hooks

Lifecycle hooks allow you to run code at specific times.